ODU Waterfield (GCP) HPC Onboarding: Learning Experience, Exercises, Debugging, and Outcomes

1. Objective and Overall Approach

My goal was to onboard onto ODU's **Waterfield** HPC cluster (hosted on **Google Cloud Platform**) and validate an end-to-end workflow suitable for machine learning research. I treated this as an engineering "smoke test" and built up capability in layers:

1. Validate login access and storage layout
2. Validate SLURM job submission and basic compute execution
3. Validate Waterfield's **container-first execution model** using modules + `crun`
4. Validate scratch I/O behavior for typical ML workloads
5. Validate GPU execution for deep learning (CUDA + PyTorch) and identify any incompatibilities

The guiding principle was to request minimal resources and keep tests short and reproducible, while staying compliant with cluster usage expectations.

2. Environment Baseline (Access + Storage + Scheduling)

Cluster access and initial environment checks

- Connected via SSH to the login node (`waterfield-login-1`) with MIDAS credentials.
- Confirmed user identity and working directory (home).
- Verified key storage locations:
  - Home: `/home/<user>` (persistent storage)
  - Scratch: `/scratch/<user>` (high-performance, not backed up)
  - Shared: `/cm/shared` (shared software/modules/data)

Scheduler visibility and partitions

Using `sinfo`, I confirmed:

- CPU partitions: `cpu-2`, `cpu-32`, `cpuflex-192`

- GPU partitions: `rtxp6000flex-*`, `h100flex-*`, `h200flex-8`, `b200flex-8`
- The health/state of partitions is dynamic (idle/mix/alloc/fail).

3. Module + Container Execution Model (Core Discovery)

Key learning: “crun” is not global; it’s provided by application modules

Initially, `crun` appeared missing (`Command not found`). This was not a permissions issue—it reflected Waterfield’s design.

- `crun` becomes available only after loading an **application module** such as:
  - `python3/2025.1-py312`
  - `pytorch-gpu/2.8.0`

So the workflow is:

1. `module load <application>`
2. then call `crun <application command>`

Critical runtime dependency: `container_env/0.1` provides `run-singularity`

I discovered `crun` is actually a wrapper around a **Singularity/Apptainer** image (`*.sif`) and relies on an RCC helper `run-singularity`. The `run-singularity` binary appears after loading:

- `module load container_env/0.1`

Required order for reliable container execution:

1. `module load container_env/0.1`
2. `module load python3/...` or `module load pytorch-gpu/...`
3. Execute via `crun ...`

This is the core “Waterfield mental model”:

> SLURM schedules the node; modules select a containerized app runtime; `crun` launches the containerized app.

4. Exercise 1: CPU Smoke Test (SLURM + `crun` + Python)

Goal

Prove that:

- SLURM job submission works
- Module load works in batch
- Container runtime launches correctly
- Python runs inside the container

What I did

Created a minimal SBATCH script targeting `cpu-2`:

- loaded `container_env/0.1`
- loaded `python3/2025.1-py312`
- executed `crun python3 -V` and a small python calculation

Issues encountered and resolution

- Early errors like `bash: : No such file or directory` occurred when I attempted to run `crun` without `container_env/0.1`.
- After enforcing the correct module load order, the CPU job produced clean output and no stderr.

Outcome

CPU smoke test succeeded end-to-end:

- Python version printed
- Linux platform identified
- simple computation performed successfully

5. Exercise 2: Scratch I/O Smoke Test (File throughput + verification + cleanup)

Goal

Validate practical scratch usage:

- create per-job scratch directory
- write sizable file efficiently
- verify contents/size with python

- cleanup

What I did

Submitted a `cpu-2` job that:

- wrote a 200MB file using `dd`
- computed SHA256 checksum inside containerized Python
- removed the scratch directory afterward

Issues encountered and resolution

- A quoting error initially passed the literal string `'$SCR'` into python, producing a FileNotFoundError.
- Fixed by passing `SCR` into the container as an environment variable (`SCR="$SCR" crun python3 ...`).

Outcome

Scratch test succeeded:

- 200MB write confirmed
- checksum and size printed
- scratch directory cleaned up

6. GPU Workflows: Provisioning Behavior + Compatibility Debugging

6.1 GPU scheduling/pending behavior and a key admin insight

While validating GPU jobs, I observed repeated state transitions like:

- `PENDING (BeginTime)`
- `CONFIGURING`
- returning to `PENDING (BeginTime)`

This looked like provisioning instability, but later I learned an important operational constraint from RCC:

> **If a GPU job requests less than ~10 minutes of wall time, a GCP provisioning quirk can cause node acquisition to fail.**

So a short test job (e.g., 5 minutes) can inadvertently trigger provisioning failure. The fix is simple:

For GPU tests, request `--time >= 00:15:00` (I used 00:20:00).
Even if the job finishes early, the requested time must be >=10 minutes to avoid triggering the known issue.

This was a major practical learning for running GPU jobs in a cloud-backed HPC cluster.

6.2 Exercise 3: H100 GPU Smoke Test (Success)

Goal

Prove:

- GPU allocation works
- container runtime sees GPU
- PyTorch can run CUDA kernels successfully

What I did

Ran a GPU smoke test on `h100flex-1` with:

- `--gres=gpu:1`
- `--time=00:20:00`
- `module load container_env/0.1`
- `module load pytorch-gpu/2.8.0`
- `nvidia-smi` + PyTorch CUDA matmul test

Outcome (clean success)

- GPU recognized: **NVIDIA H100 80GB HBM3**
- PyTorch: `2.8.0+cu126`
- CUDA: `12.6`
- Capability: `(9,0)` (sm_90)
- CUDA matmul executed successfully

Conclusion: Waterfield's GPU + container + PyTorch stack works correctly on H100.

6.3 Exercise 4: RTX PRO 6000 "Blackwell" (Failure explained by architecture mismatch)

Although I do not need RTX today, I validated and diagnosed the failure mode because it's a valuable learning outcome.

Observation

On `rtxp6000flex-1`, `nvidia-smi` showed the GPU as:

- **NVIDIA RTX PRO 6000 Blackwell Server Edition**
- Capability `(12,0)` (sm_120)

PyTorch module used:

- `pytorch-gpu/2.8.0+cu126` (CUDA 12.6)

Failure mode

PyTorch emitted warnings that the installed build supports architectures only up to **sm_90**, and then crashed when launching a CUDA kernel:

- `CUDA error: no kernel image is available for execution on the device`

Root cause

This is a classic **binary compatibility** issue:

- Blackwell GPUs use sm_120
- The existing PyTorch module was not compiled with sm_120 kernels
- Therefore, CUDA kernel launch fails even though the driver and GPU are healthy

Conclusion: RTX PRO 6000 nodes are healthy and visible, but the current `pytorch-gpu/2.8.0+cu126` module is not compatible with Blackwell (sm_120). A newer PyTorch build (typically CUDA 12.8/12.9 and sm_120-enabled) would be required for RTX-based PyTorch workloads.

7. Key Lessons Learned (What I would reuse going forward)

A) Waterfield execution model

- Waterfield is not a “traditional” bare-metal module environment. It is **containerized by default**.
- The correct mental model is:
  - SLURM allocates compute
  - Lmod selects the containerized application runtime
  - `crun` launches the `.sif` container using `run-singularity`

B) Required module order

For consistent success:

1. `module load container_env/0.1`

2. `module load <application module>`
3. Run commands via `crun ...`

C) GPU walltime quirk (cloud provisioning)

- For GPU jobs, request **>=10 minutes** walltime (I use 20 minutes for tests), even for small smoke tests.

D) Hardware/software compatibility matters (especially with new GPUs)

- H100 (sm_90) works with the existing PyTorch build.
- Blackwell (sm_120) requires a newer PyTorch build compiled for that architecture.

8. Final Status

- ✅ CPU jobs (SLURM + container runtime + python) validated
- ✅ Scratch I/O validated (write, checksum, cleanup)
- ✅ H100 GPU jobs validated (CUDA + PyTorch matmul success)
- ⚠️ RTX PRO 6000 Blackwell PyTorch workloads currently incompatible with the provided PyTorch module due to sm_120 support requirements (driver/GPU healthy; PyTorch build mismatch)

9. Relevance to AI in Healthcare (Why this matters)

For clinical ML workflows, a reliable HPC environment enables:

- rapid experimentation (model training, ablations)
- scalable GPU runs for deep learning
- reproducible environments via containerized modules (reducing “works on my machine” issues)
- clear operational practices for data handling (home vs scratch, cleanup, reproducibility)

Waterfield’s cloud-backed model adds new operational considerations (provisioning time, walltime constraints) that are directly relevant to real-world cloud ML systems (including Vertex AI-style job scheduling and runtime packaging).

10. Next steps: using Waterfield to execute my CS781 course project experiments

- I will use Waterfield (SLURM + `crun`) as the reproducible execution environment for my full experiment matrix: run BioMistral-7B inference for PubMed RCT (and MedNLI if available) across **FP16 vs INT8 vs INT4 (NF4 via bitsandbytes)**, extract single-token code probabilities/logits, and compute **performance + calibration (ECE/reliability)** and **prompt-robustness** metrics; I'll run the GPU-heavy jobs on **H100/H200/B200 partitions** (where PyTorch is already confirmed working) and keep results/logs organized via scratch staging + persistent home artifacts so every figure/table in the final report is traceable to a specific Waterfield job configuration.